SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA17**

COURSE OUTCOME COVERED

CO3: *Analyze knowledge representation and reasoning using propositional logic, FOL, inference, resolution, chaining methods, knowledge representation to perform logical reasoning for drawing valid conclusions and decision making. (BL4)*

Assessment Tool Weightage: 50%

Time Duration: 1 Hour

QUESTIONS: 4 **Total Marks:40**

Annexure A

CASE STUDY

Code of Conduct:

I ____________________(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Annexure A

CASE STUDY

Case Study 1 — Smart Medical Diagnosis System

A hospital is designing a rule-based Logical Agent for preliminary patient diagnosis. The system's knowledge base contains:

I. If a patient has Fever AND Cough → Possible Flu
Ii. If a patient has Fever AND Rash → Possible Measles
Iii. If a patient has Cough AND Breathlessness → Possible Pneumonia
Iv. Facts about Patient A: Fever = True, Cough = True, Breathlessness = True

(a) Represent the above rules and facts precisely using Propositional Logic symbols. Define each symbol used.

(b) Apply Forward Chaining on the knowledge base to derive all possible diagnoses for Patient A. Show each inference step with the rule fired and fact derived.

(c) The goal is to confirm 'Patient A has Pneumonia.' Apply Backward Chaining and draw the goal reduction tree to verify this goal.

Case Study 2 — Autonomous Traffic Management Agent

A smart city traffic controller uses a FOL-based knowledge base to manage emergency vehicle routing:

I. ∀x: Vehicle(x) ∧ EmergencyType(x) → ClearPath(x) [Rule 1]
II. ∀x: ClearPath(x) → GreenSignal(x) ∧ ¬RedSignal(x) [Rule 2]
III. ∀x ∀y: Vehicle(x) ∧ Behind(x,y) ∧ GreenSignal(y) → Proceed(x) [Rule 3]
IV. Facts: Vehicle(Ambulance), EmergencyType(Ambulance), Vehicle(CarA), Behind(CarA, Ambulance)

(a) Apply Unification to match Rule 1 with the given facts. Show the complete substitution set θ (MGU) at each step.

(b) Using the Resolution method, prove that 'Proceed (CarA)' is true. Convert relevant clauses to CNF and show each resolution step.

(c) Analyze whether the current FOL knowledge base is sufficient to handle two simultaneous emergency vehicles. Identify what additional rules or facts are needed and justify logically.

Case Study 3 — Agricultural Expert Reasoning System

An agricultural advisory logical agent operates with the following propositional knowledge base (KB):

1) P1: SoilDry → IrrigationNeeded
2) P2: IrrigationNeeded ∧ CropWheat → ApplyDripMethod
3) P3: ¬ApplyDripMethod ∧ CropWheat → CropAtRisk
4) Facts: SoilDry = True, CropWheat = True

(a) Convert all sentences P1, P2, P3 and the facts into Conjunctive Normal Form (CNF). Show each conversion step (eliminate implication → move ¬ inward → distribute).

(b) Using Resolution Refutation, prove the theorem 'ApplyDripMethod'. Negate the goal, combine with the KB in CNF, and derive the empty clause □. Draw the resolution proof tree.

(c) If the fact 'SoilDry' is removed from the KB, analyze step-by-step how logical conclusions change. Does 'CropAtRisk' still follow? Apply appropriate reasoning patterns to justify.

Case Study 4 — Wumpus World Knowledge Agent

An AI agent navigating the Wumpus World perceives: Stench at [1,2], Breeze at [1,1], Glitter at [2,2]. The agent uses a propositional KB with these rules:

1) Stench [1,2] → Wumpus is adjacent to cell [1,2]
2) Breeze [1,1] → Pit is adjacent to cell [1,1]
3) Glitter [x,y] → Gold is at [x,y]
4) ¬Wumpus [x,y] ∧ ¬Pit[x,y] → Safe[x,y]

(a) Construct the agent's belief state from its percepts. Apply Modus Ponens systematically to derive new knowledge facts. List all derived conclusions.

(b) Using Forward Chaining from the agent's percepts, identify the possible locations of the Wumpus and Pit. Show each reasoning step precisely.

(c) Evaluate whether the path [1,1] → [1,2] → [2,2] is safe for the agent to collect the gold. Use the knowledge derived in (a) and (b) to justify your conclusion with logical inference.

Case Studies on Logical Agents — Complete Solutions

Case Study 1 — Smart Medical Diagnosis System

(a) Representation using Propositional Logic

Define the propositional symbols:

- F = Patient has Fever
- C = Patient has Cough
- R = Patient has Rash
- B = Patient has Breathlessness
- Flu = Patient has Possible Flu
- Measles = Patient has Possible Measles
- Pneumonia = Patient has Possible Pneumonia

Rules:

1. Fever AND Cough → Possible Flu
   $F \land C \rightarrow Flu$
2. Fever AND Rash → Possible Measles
   $F \land R \rightarrow Measles$
3. Cough AND Breathlessness → Possible Pneumonia
   $C \land B \rightarrow Pneumonia$

Facts about Patient A:

- F = True
- C = True
- B = True

Therefore:

KB = {F, C, B, $F \land C \rightarrow Flu$, $F \land R \rightarrow Measles$, $C \land B \rightarrow Pneumonia$}

(b) Forward Chaining

Forward Chaining starts with known facts and applies rules whose premises are satisfied.

Initial facts:

- Fever(A) = True
- Cough(A) = True
- Breathlessness(A) = True

Step 1: Apply Rule 1

Rule:

Fever ∧ Cough → Flu

Since Fever = True and Cough = True:

Fever(A) ∧ Cough(A) → Flu(A)

Therefore:

Flu(A) = True

Step 2: Apply Rule 2

Rule:

Fever ∧ Rash → Measles

We know Fever(A) = True, but Rash(A) is not given.

Therefore, Rule 2 cannot fire.

Step 3: Apply Rule 3

Rule:

Cough ∧ Breathlessness → Pneumonia

Since Cough(A) = True and Breathlessness(A) = True:

Cough(A) ∧ Breathlessness(A) → Pneumonia(A)

Therefore:

Pneumonia(A) = True

Final conclusions

- Possible Flu(A) = True
- Possible Measles(A) = Not derivable
- Possible Pneumonia(A) = True

(c) Backward Chaining for Pneumonia

Goal:

Pneumonia(A)

Find a rule whose conclusion is Pneumonia:

Cough(x) ∧ Breathlessness(x) → Pneumonia(x)

Goal reduction:

```
            Goal: Pneumonia(A)
                  |
                  | Rule 3
                  v
         Cough(A) ∧ Breathlessness(A)
              /          \
             /            \
            v              v
       Cough(A) = True    Breathlessness(A) = True
            \            /
             \          /
              \        /
               ---- TRUE ---
```

Both subgoals are facts in the KB.

Therefore:

Pneumonia(A) is proved by Backward Chaining.

Case Study 2 — Autonomous Traffic Management Agent

(a) Unification

Rule 1:

∀x [Vehicle(x) ∧ EmergencyType(x) → ClearPath(x)]

Given facts:

- Vehicle(Ambulance)
- EmergencyType(Ambulance)

We match the rule variables with the facts.

Step 1

Match:

Vehicle(x) with Vehicle(Ambulance)

Substitution:

θ_1 = {x/Ambulance}

Step 2

Apply θ_1 to the second condition:

EmergencyType(x)

becomes:

EmergencyType(Ambulance)

This matches the given fact.

Therefore:
θ_2 = {x/Ambulance}
The Most General Unifier is:
MGU θ = {x/Ambulance}
Applying this substitution to the conclusion:
ClearPath(x)
gives:
ClearPath(Ambulance)

(b) Resolution Proof of Proceed(CarA)
Relevant facts:

1. Vehicle(Ambulance)
2. EmergencyType(Ambulance)
3. Vehicle(CarA)
4. Behind(CarA, Ambulance)

Rules:
R1:
Vehicle(x) ∧ EmergencyType(x) → ClearPath(x)
CNF:
¬Vehicle(x) ∨ ¬EmergencyType(x) ∨ ClearPath(x)
R2:
ClearPath(x) → GreenSignal(x)
CNF:
¬ClearPath(x) ∨ GreenSignal(x)
R3:
Vehicle(x) ∧ Behind(x,y) ∧ GreenSignal(y) → Proceed(x)
CNF:
¬Vehicle(x) ∨ ¬Behind(x,y) ∨ ¬GreenSignal(y) ∨ Proceed(x)
We want to prove:
Proceed(CarA)
Negate the goal:
¬Proceed(CarA)
Resolution Step 1
From R1 and the facts:
Vehicle(Ambulance)
EmergencyType(Ambulance)
Resolve:
¬Vehicle(Ambulance) ∨ ¬EmergencyType(Ambulance) ∨ ClearPath(Ambulance)
Result:
ClearPath(Ambulance)
Resolution Step 2
Use R2:
¬ClearPath(Ambulance) ∨ GreenSignal(Ambulance)
Resolve with:
ClearPath(Ambulance)
Result:
GreenSignal(Ambulance)
Resolution Step 3

R3 instantiated with x = CarA and y = Ambulance:

¬Vehicle(CarA) ∨ ¬Behind(CarA,Ambulance) ∨ ¬GreenSignal(Ambulance) ∨ Proceed(CarA)

Using:

- Vehicle(CarA)
- Behind(CarA, Ambulance)
- GreenSignal(Ambulance)

we derive:

Proceed(CarA)

Resolution Step 4 — Contradiction

We also have the negated goal:

¬Proceed(CarA)

Resolve:

Proceed(CarA)

with:

¬Proceed(CarA)

Result:

□

Therefore, the empty clause is derived.

Hence:

KB ⊨ Proceed(CarA)

So Proceed(CarA) is logically proved.

(c) Two simultaneous emergency vehicles

The current KB can handle more than one emergency vehicle in principle, because Rule 1 is universally quantified:

∀x [Vehicle(x) ∧ EmergencyType(x) → ClearPath(x)]

For example, if there are two ambulances:

- Vehicle(Ambulance1)
- EmergencyType(Ambulance1)
- Vehicle(Ambulance2)
- EmergencyType(Ambulance2)

then Rule 1 derives:

- ClearPath(Ambulance1)
- ClearPath(Ambulance2)

However, the KB does not contain enough information to coordinate competing emergency routes.

Additional facts/rules would be useful, such as:

Rule 4:

Emergency vehicles have different priorities:

EmergencyPriority(x) ∧ HigherPriority(x,y) → GivePriority(x,y)

Rule 5:

Only one conflicting traffic direction gets a green signal:

ConflictingPath(x,y) ∧ GivePriority(x,y) → GreenSignal(x) ∧ ¬GreenSignal(y)

Additional facts should identify:

- Location of each emergency vehicle
- Destination of each emergency vehicle
- Which paths conflict
- Priority between emergency vehicles

- Traffic signal locations

Therefore, the existing KB supports independent emergency vehicles but needs conflict-resolution and priority rules for simultaneous emergencies.

Case Study 3 — Agricultural Expert Reasoning System

(a) CNF Conversion

P1

Original:

SoilDry → IrrigationNeeded

Eliminate implication:

¬SoilDry ∨ IrrigationNeeded

This is already CNF.

P1 = ¬SoilDry ∨ IrrigationNeeded

P2

Original:

IrrigationNeeded ∧ CropWheat → ApplyDripMethod

Eliminate implication:

¬(IrrigationNeeded ∧ CropWheat) ∨ ApplyDripMethod

Apply De Morgan's law:

¬IrrigationNeeded ∨ ¬CropWheat ∨ ApplyDripMethod

This is CNF.

P2 = ¬IrrigationNeeded ∨ ¬CropWheat ∨ ApplyDripMethod

P3

Original:

¬ApplyDripMethod ∧ CropWheat → CropAtRisk

Eliminate implication:

¬(¬ApplyDripMethod ∧ CropWheat) ∨ CropAtRisk

Move negation inward:

ApplyDripMethod ∨ ¬CropWheat ∨ CropAtRisk

This is CNF.

P3 = ApplyDripMethod ∨ ¬CropWheat ∨ CropAtRisk

Facts

SoilDry = True

CNF:

SoilDry

CropWheat = True

CNF:

CropWheat

Complete CNF KB

1. ¬SoilDry ∨ IrrigationNeeded
2. ¬IrrigationNeeded ∨ ¬CropWheat ∨ ApplyDripMethod
3. ApplyDripMethod ∨ ¬CropWheat ∨ CropAtRisk
4. SoilDry
5. CropWheat

(b) Resolution Refutation for ApplyDripMethod

The theorem to prove is:

ApplyDripMethod

Negate the goal:

¬ApplyDripMethod

Add this to the KB.

Relevant clauses:

1. ¬SoilDry ∨ IrrigationNeeded
2. ¬IrrigationNeeded ∨ ¬CropWheat ∨ ApplyDripMethod
3. SoilDry
4. CropWheat
5. ¬ApplyDripMethod ← Negated goal

Resolution Step 1

Resolve:

¬SoilDry ∨ IrrigationNeeded

with:

SoilDry

Result:

IrrigationNeeded

Resolution Step 2

Resolve:

¬IrrigationNeeded ∨ ¬CropWheat ∨ ApplyDripMethod

with:

IrrigationNeeded

Result:

¬CropWheat ∨ ApplyDripMethod

Resolution Step 3

Resolve:

¬CropWheat ∨ ApplyDripMethod

with:

CropWheat

Result:

ApplyDripMethod

Resolution Step 4

Resolve:

ApplyDripMethod

with the negated goal:

¬ApplyDripMethod

Result:

□

Therefore, the empty clause is obtained.

Hence:

KB ⊨ ApplyDripMethod

Resolution proof tree

```
          □
         /  \
```

```
                 /     \
       ApplyDripMethod     ¬ApplyDripMethod
             |
      ----------------
      |             |
CropWheat     ¬CropWheat ∨ ApplyDripMethod
                    |
              ----------------
              |             |
      IrrigationNeeded   ¬IrrigationNeeded
                           ∨
                 ¬CropWheat ∨ ApplyDripMethod
                           |
                   IrrigationNeeded
                           |
               ¬SoilDry ∨ IrrigationNeeded
                           |
                        SoilDry
```

Thus, ApplyDripMethod is proved.

(c) Removing SoilDry

Suppose SoilDry is removed.

We still have:

CropWheat

But we no longer have:

SoilDry

Therefore, Rule P1:

SoilDry → IrrigationNeeded

cannot be fired.

So:

IrrigationNeeded cannot be derived.

Since P2 requires:

IrrigationNeeded ∧ CropWheat

we cannot derive:

ApplyDripMethod

Also, P3 requires:

¬ApplyDripMethod ∧ CropWheat → CropAtRisk

We know CropWheat, but we do not know:

¬ApplyDripMethod

Therefore:

CropAtRisk cannot be derived.

Important logical point:

Failure to prove ApplyDripMethod does not mean ¬ApplyDripMethod.

Therefore, after removing SoilDry:

- IrrigationNeeded → Not derivable
- ApplyDripMethod → Not derivable
- CropAtRisk → Not derivable

So CropAtRisk does not follow from the remaining KB.

Case Study 4 — Wumpus World Knowledge Agent

(a) Belief State and Modus Ponens

Initial percepts:

1. Stench[1,2]
2. Breeze[1,1]
3. Glitter[2,2]

Rules:

1. Stench[1,2] → Wumpus is adjacent to [1,2]
2. Breeze[1,1] → Pit is adjacent to [1,1]
3. Glitter[x,y] → Gold[x,y]
4. ¬Wumpus[x,y] ∧ ¬Pit[x,y] → Safe[x,y]

Step 1 — Stench

Fact:

Stench[1,2]

Rule:

Stench[1,2] → Wumpus adjacent to [1,2]

Therefore:

Wumpus may be in an adjacent cell to [1,2].

Possible adjacent cells:

- [1,1]
- [1,3]
- [2,2]

Step 2 — Breeze

Fact:

Breeze[1,1]

Rule:

Breeze[1,1] → Pit adjacent to [1,1]

Possible adjacent cells:

- [1,2]
- [2,1]

Step 3 — Glitter

Fact:

Glitter[2,2]

Rule:

Glitter[x,y] → Gold[x,y]

Therefore:

Gold[2,2]

Step 4 — Safe cells

Rule:

¬Wumpus[x,y] ∧ ¬Pit[x,y] → Safe[x,y]

No explicit negative facts about Wumpus or Pit locations are provided.

Therefore, Safe[x,y] cannot be derived using Rule 4.

Derived knowledge

- Wumpus is possible adjacent to [1,2]
- Pit is possible adjacent to [1,1]

- Gold is at [2,2]
- No cell can currently be proven safe using Rule 4

(b) Forward Chaining
Start with:
Stench[1,2], Breeze[1,1], Glitter[2,2]
Forward-chaining Step 1
Stench[1,2]
fires Rule 1:
Wumpus adjacent to [1,2]
Possible Wumpus locations:
{[1,1], [1,3], [2,2]}
Forward-chaining Step 2
Breeze[1,1]
fires Rule 2:
Pit adjacent to [1,1]
Possible Pit locations:
{[1,2], [2,1]}
Forward-chaining Step 3
Glitter[2,2]
fires Rule 3:
Gold[2,2]
Step 4
Rule 4 requires explicit negative information:
¬Wumpus[x,y] ∧ ¬Pit[x,y]
No such facts are available.
Therefore, no additional Safe facts can be derived.
Final possible locations

Object	Possible locations
Wumpus	[1,1], [1,3], [2,2]
Pit	[1,2], [2,1]
Gold	[2,2]

(c) Safety of Path [1,1] → [1,2] → [2,2]
The proposed path is:
[1,1] → [1,2] → [2,2]
We know:

- There is a Breeze at [1,1]
- Therefore, a Pit may be at [1,2] or [2,1]
- There is a Stench at [1,2]
- Therefore, a Wumpus may be at [1,1], [1,3], or [2,2]
- Gold is at [2,2]

Therefore:
[1,2] is a possible Pit location.
Also:
[2,2] is a possible Wumpus location.
The KB does not provide enough information to prove:

¬Pit[1,2]
or
¬Wumpus[2,2]
Hence, Rule 4 cannot establish these cells as safe.
Conclusion
The path:
[1,1] → [1,2] → [2,2]
cannot be confirmed as safe.
In fact, according to the current knowledge:

- [1,2] may contain a Pit
- [2,2] may contain the Wumpus
- [2,2] definitely contains Gold

Therefore, the logical agent should not assume that this path is safe without obtaining additional percepts or knowledge.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly
Application of Concepts	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	10	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand